TASK MANAGEMENT SYSTEM

Implemented Using the C Programming Language

Data Structures Project • Abstract (Page 1 of 3)

1. Introduction

The Task Management System is a menu-driven console application developed using the C programming language. It helps users create, view, update, complete, and delete tasks in one organized place. Each task can contain a title, description, priority, due date, and completion status. It is designed as a practical Data Structures project without requiring a website or database server.

2. Problem Statement

Managing several personal or academic tasks through scattered notes can make it difficult to remember deadlines and identify unfinished work. A simple application is needed to keep task details organized, display pending and completed work, and provide operations for updating task information. This project addresses the need using C and suitable data structures.

3. Objectives

- Add and store new tasks with relevant details.
- Display all tasks and filter pending or completed tasks.
- Update task details and mark tasks as completed.
- Delete tasks when they are no longer required.
- Organize tasks by priority or due date where supported.
- Demonstrate linked lists, queues, stacks, and trees using C.

4. Methodology

4.1 System Workflow

The program displays a menu. The user chooses an operation, enters task details or a task ID, and the program updates the task collection. The menu repeats until Exit is selected. Each operation is implemented as a separate C function, keeping the program modular and easier to test.

4.2 Implementation Approach

Structures (struct) represent tasks, while functions perform each operation. A linked list can store a flexible number of tasks. A queue can hold tasks waiting to be processed, a stack can maintain recent-action history, and a tree can organize tasks by priority or category if these features are implemented.

TASK MANAGEMENT SYSTEM

Implemented Using the C Programming Language

Data Structures Project • Abstract (Page 2 of 3)

5. System Design

5.1 Overall Design

The application consists of a console menu, task-management functions, and an in-memory collection of task records. Each task may contain a unique ID, title, description, priority, due date, and status. File handling can optionally be added to save and load tasks between sessions.

Console Menu User choices	**C Functions** Add, view, edit, complete, delete	**Task Records** Structures and data structures

5.2 Menu Workflow

Start program
Display task-management menu
Read and validate user's choice
Perform selected operation
Show result and return to menu
Exit when requested

5.3 Data Structures Used

• **Linked list:** stores a flexible number of task records.

• **Queue:** handles tasks waiting in FIFO order.

• **Stack:** supports recent-action history or undo.

• **Tree:** organizes tasks by priority or category.

6. Features

- Add tasks with title, description, priority, and due date.
- View all tasks and search by task ID.
- Edit task information and update status.
- Mark tasks completed and list pending tasks.
- Delete tasks that are no longer needed.
- Optional sorting and file-based saving.

7. Sample System Scenario

A user selects Add Task, enters “Complete Data Structures assignment,” chooses a priority and due date, and saves it. The task appears in the list with Pending status. After finishing the work, the user selects its ID and marks it Completed.

TASK MANAGEMENT SYSTEM

Implemented Using the C Programming Language

Data Structures Project • Abstract (Page 3 of 3)

8. Example Output

Illustrative task list:

ID	Task	Priority	Status
1	Complete DS assignment	High	Pending
2	Read C programming chapter	Medium	Completed

Example menu:

```
1. Add Task
2. View All Tasks
3. Search Task
4. Update Task
5. Mark Task Completed
6. Delete Task
7. View Pending Tasks
8. Exit
```

9. Algorithms in Detail

9.1 Linked List Operations

To add a task, allocate a new node, store the task fields, and link the node into the list. To find or update a task, traverse the list and compare IDs. To delete a task, locate the node, adjust the previous node's link or the head pointer, and free the removed node.

9.2 Queue and Stack Operations

A queue stores task IDs waiting for processing: insertion occurs at the rear and removal at the front. A stack stores recent actions: actions are pushed onto the top and retrieved in last-in, first-out order. Include these features only if implemented in the final program.

10. Results and Discussion

The proposed C application provides a structured way to record tasks and track their status through a simple menu. Its core operations help users distinguish pending tasks from completed work. Structures and linked lists demonstrate dynamic memory management, while optional queues, stacks, and trees extend the project to cover additional data-structure concepts. Persistent storage and advanced sorting depend on the final implementation.

11. Conclusion

The Task Management System implemented in C is a practical console-based project for organizing daily or academic tasks. It supports task creation, viewing, updating, completion tracking, and deletion. By applying fundamental data structures, the project connects theoretical concepts with a useful program while keeping the implementation focused on C.

Keywords

C Programming, Task Management, Data Structures, Linked List, Queue, Stack, Tree, Task Tracking, Console Application.